Agentic SDD Verification Handbook

This handbook describes the repository-native verification workflow. The accepted feature specification, plan, schemas, profile and python3 tooling/agent-harness/verify.py --help remain authoritative when details differ. This guide does not grant lifecycle, launch, retry, commit or remote-write authority.

Status and safety boundary

Verification v2 is integrated into the repository's planning and task workflow, but the canonical M5.3 Design Gate is NOT PASS. Do not describe the overall design gate or an unqualified backend as accepted or production-ready. An advisory plan is not an accepted plan. Execution requires an exact lifecycle-accepted plan and a backend qualified for its required protections; if either is missing, execution fails closed.

The harness may inspect local Git state and create task-local checkpoints required by the accepted lifecycle. It does not push, open pull requests, merge into a user branch, deploy, or mutate remote trackers. Task/product changes remain under human-controlled Git workflow.

Choose the workflow

Use the lightest flow that fits the uncertainty:

Workflow choices (work | flow | verification)

Work: Small and well understood; Flow: spec to tasks to orchestrate; Verification: Existing deterministic gates and independent evaluator.

Work: Medium with a few uncertainties; Flow: spec to design to [prototype] to verification contract to tasks; Verification: Resolve material design questions and add independent criteria for meaningful risk.

Work: Large, foggy, or architectural; Flow: wayfinder to reconcile to spec to design/grill to [prototype] to verification contract to tasks; Verification: Close blocking assumptions, preserve decision history, then derive and independently evaluate implementation.

Prototype code is disposable evidence. Translate accepted findings into the plan and normal task DAG; do not promote a scratch prototype directly.

Verification modes and task completion

The verifier has two immutable origin policies:

- task-completion is the outer completion boundary. After builder success, the runner freshly executes every declared verification command occurrence and its required dependency closure against the sealed candidate. A repeated command is still a distinct occurrence. Cached integration evidence cannot replace this execution.
- integration is an explicit verification workflow that may reuse eligible evidence, but only within the exact authoritative plan and execution identity. A continuation inherits that identity; it is not a new plan or a shortcut around task completion.

Use verify.py plan to inspect an advisory plan. Its three decisions are:

- ALREADY_GREEN: complete terminal PASS evidence exists for the current fingerprint and required artifacts; action is REUSE only where the accepted origin policy allows it.
- INVALIDATED_BY_THIS_PATCH: prior PASS exists but an input, policy, dependency or artifact changed; action is always RUN.
- RUN_NOW: no eligible PASS exists, or policy requires a fresh execution; action is RUN.

INVALIDATED_BY_THIS_PATCH is explanatory, never a skip. Required gates run in deterministic dependency order. A failure blocks dependent and not-yet-run required gates; it does not turn them green.

Create an advisory plan with trusted values from the active workflow:

```
python3 tooling/agent-harness/verify.py plan \
  --repo . \
  --profile showcase \
  --base-sha <full-base-commit-sha> \
  --family-id <stable-family-id> \
  --policy-checkpoint <accepted-policy-checkpoint>
```

The logical profile ID resolves under the trusted tooling/agent-harness/verification-profiles/ directory. Do not pass a provider-worktree profile path or invent identity values. Planning grants no launch authority.

Execute only the plan already accepted by the lifecycle authority:

```
python3 tooling/agent-harness/verify.py run \
  --mode integration \
```

```
  --repo . \
  --plan-id <exact-accepted-verification-plan-id>
```

An exact --unit-id may further select a unit in that plan. --failure-grant GATE=GRANT references an already-issued grant; it does not issue one. Missing --repo or --plan-id is blocked. Consult command help for the checked-out version. grant-import changes local control state and is not a smoke-test command.

Fingerprints, reuse, and artifacts

A reusable result is bound to the exact trusted profile and policy, gate and command bytes, working directory, origin policy, required obligation identity, sandbox and retry controls, matched input set and content identities, tool probes, dependencies, artifacts and execution context. File additions, deletions, staged/unstaged/untracked changes and relevant symlink identity are accounted for. Symlink-affected or opaque external inputs are non-cacheable or invalid policy under the accepted rules.

The result is reusable only when the immutable terminal evidence is complete, its bindings match the active plan, and pre-run and post-run observations agree. A changed input invalidates prior PASS. A consumer cannot reuse an artifact if its required producer evidence is missing or its content identity changed. Cache indexes and summaries are derived data: corruption is ignored/rebuilt and safe work runs fresh. Attempt, failure, grant and manual-evidence records are audit history and are not erased as cache cleanup.

Version-1 provenance remains readable for reporting, but never authorizes v2 reuse. Unknown evidence versions cannot authorize reuse. Unknown lifecycle protocol fields or versions fail closed; an older harness must not reinterpret, rewrite, clean or migrate newer .agent-state authority.

Failure, retry, and resume

The first required failure stops dependent execution and is preserved. Critical failures create a durable fingerprint-scoped fence. A new run ID, task attempt or plan does not itself clear a matching fence. Same-fingerprint retry requires an exact, authenticated, one-shot critical-gate retry grant from the trusted human issuer registry; automation cannot mint or sign it. Manual review evidence is never retry authorization.

An interrupted command is not presumed safe to relaunch. The supervisor and backend must prove drainage of the command and descendants. Uncertain ownership quarantines the worktree and blocks repository-wide admission until the trusted recovery path resolves it. Lease expiry or PID absence alone is not drainage proof. Resume may continue only the exact accepted authority and recovery slot; a changed plan, candidate, surface, policy or other bound identity requires fresh authority and evidence. Prior FAIL records remain visible after a later authorized PASS.

Use the supported admission recovery entry point:

```
python3 tooling/agent-harness/harness.py recover-verification --repo .
```

It releases only proven-dead prestart reservations and authoritative recoverable journals. It does not clear every quarantine. Active or uncertain ownership, or a journal still requiring backend drainage, remains blocked (exit 6). Never manually delete unresolved started.json, drainage, ownership or quarantine records; lease expiry and PID absence do not prove safe release.

Typical outcomes include needs-human (exit 4) for a required human decision and verification-blocked (exit 5) for missing or unsupported authority/capability. Read-only diagnostics and advisory plans cannot clear either condition. Follow the exact machine outcome and remediation in the current CLI and accepted spec; do not retry blindly.

Telemetry and privacy

Telemetry distinguishes provider runs, task attempts, verification families/units and gate outcomes, including planned, executed, reused, invalidated, blocked and failed. Running and abandoned records do not count as successes. Missing chronology, incomplete legacy lineage or unknown usage remains UNKNOWN; cost is never inferred from token counts. "Saved time" and "saved cost" are omitted unless a future report explicitly labels and documents an estimate.

Structured reports use an allowlist and exclude raw prompts, environment contents, command output, report bodies and seeded secrets. Raw execution logs are diagnostic data, not reusable evidence; deleting them does not invalidate complete terminal evidence. Local telemetry is not lifecycle authority.

Manual reviewer/evaluator evidence

Manual registration records provenance; it does not launch a provider, change task/design state,

authorize retry, or satisfy an obligation by itself. Registration requires the exact report snapshot and a canonical signed Ed25519 manual-review-attestation-v1 from a trusted, enabled, non-revoked issuer explicitly allowed for manual-review. The reviewer, report hash, verdict, checkpoint and any task/attempt/plan/obligation/candidate/surface bindings must match trusted records. .agent-state alone accepts plan-obligation coverage through its coverage transaction, after re-verifying the signed bytes. Caller labels such as --by or --operator are not identity proof. The signing key stays outside automation.

After a manual reviewer or evaluator result is used as accepted evidence, show this instruction:

```
MANUAL TELEMETRY REQUIRED
```

Register it with the exact verified values and files; do not paste report contents into telemetry:

```
python3 tooling/agent-harness/telemetry.py --repo . record-manual \
  --feature <feature-id> \
  --role <reviewer|evaluator|architecture-reviewer|verification-author> \
  --provider <provider-name> \
  --checkpoint <full-commit-id> \
  --verdict <PASS|FAIL|NEEDS-HUMAN> \
  --report <path-to-final-report> \
  --attestation <path-to-signed-attestation> \
  [--task <task-id> --task-attempt <attempt>] \
  [--plan-id <accepted-plan-id>]
```

Use task, attempt and plan flags only when that exact scope is present in the trusted attestation and lifecycle. An empty/revoked registry, invalid signature, stale checkpoint, mismatched identity or unavailable coverage authority means registration/coverage is not accepted. Do not substitute a file, persona name, provider statement or physical review for a signature. For SDD-OBS-001, all qualifying manual reviews/evaluations after this command became available must be registered for complete observability; omissions leave telemetry partial and block that observability DoD, without rewriting the review or authorizing a workflow transition.

Rollback and backend limits

Runtime reporting compatibility is not permission to downgrade lifecycle state. V1 records can be read for their limited reporting fields but cannot authorize v2 reuse. Runtime caches can be discarded; immutable attempt, failure, grant and manual-evidence history cannot be rewritten as cleanup. Unknown lifecycle schema/protocol, active reservation/consumption, or unsupported authority causes lifecycle-dependent commands to fail closed without mutation. Only a compatible harness may perform an explicit lossless forward migration under the lifecycle lock and expected-generation CAS.

The current M5.3 Design Gate is not PASS. The native macOS backend has not qualified strong descendant containment; the Docker Desktop investigation does not by itself qualify the current adapter for M5.3 launches. A backend must prove protected-path enforcement and descendant drainage for its exact target and policy. Until the accepted qualification report binds those checks, execution remains blocked. No degraded auto/off mode, Docker capability label, or test image substitutes for qualification.

Helpful references

- Agentic SDD overview (README.md)
- Current verification operator reference (verification-operator-reference.md)
- Accepted SDD-OBS-001 specification (../specs/SDD-OBS-001/spec.md)
- Accepted SDD-OBS-001 plan (../specs/SDD-OBS-001/plan.md)
- Agentic SDD constitution (constitution.md)